

20250729日报

今日工作内容

1. 在sgameproto后置流水线中添加桩代码的生成，并完成测试；
最终脚本如下：

```
1  source ~/.bashrc
2
3  cd $WORKSPACE
4  echo ${BK_CI_HOOK_TARGET_BRANCH}
5
6  git fetch origin ${BK_CI_HOOK_TARGET_BRANCH}
7  git checkout ${BK_CI_HOOK_TARGET_BRANCH}
8
9  create_init_py() {
10     local dir="$1"
11     init_file="${dir}/__init__.py"
12     if [ ! -f "$init_file" ]; then
13         touch "$init_file"
14         echo "Created: $init_file"
15     fi
16 }
17
18 git diff --name-only HEAD^ HEAD | while read file; do
19     dirname "$file"
20 done | sort -u | grep -v "^. $" | while read dir; do
21     echo "$dir"
22     make dir="$dir"
23     create_init_py "$dir"
24     if find "$dir" -maxdepth 1 -name "*.proto" | grep -q .; then
25         echo "Found .proto files in $dir, running trpc create"
26         find "$dir" -maxdepth 1 -name "*.proto" | while read proto_file; do
27             echo "$proto_file"
28             output_dir="${dir}"
29             trpc create --rpconly \
30                 --nogomod \
31                 --protofile="$proto_file" \
32                 --output="$output_dir"
33         done

```

在自己的helloworld测试结果如下：

流水线端：

```
1 master
2 来自 http://git.woa.com/drewjlli/helloworld
3 * branch master -> FETCH_HEAD
4 您的分支与上游分支 'origin/master' 一致。
5 已经位于 'master'
6 sgameproto/ai_play_model2
7 for x in $(find ./${dir} -name *.proto); do protoc --go_out=paths=source_relative:. ${x}; done
8 for x in $(find ./${dir} -name *.proto); do protoc --python_out=. --mypy_out=. ${x}; done
9 Writing mypy to sgameproto/ai_play_model2/lobby_require_invite/lobby_require_invite_pb2.pyi
10 Writing mypy to sgameproto/ai_play_model2/ai_play_model_pb2.pyi
11 Created: sgameproto/ai_play_model2/__init__.py
12 Found .proto files in sgameproto/ai_play_model2, running trpc create
13 sgameproto/ai_play_model2/ai_play_model.proto
14 [create] protoc-gen-secv-v2 is installed to /root/go/bin, please ensure that it is added to PATH environment variable.
15 [create] goimports is installed to /root/go/bin, please ensure that it is added to PATH environment variable.
16 [create] mockgen is installed to /root/go/bin, please ensure that it is added to PATH environment variable.
17 [create] protoc-gen-go-vtproto is installed to /root/go/bin, please ensure that it is added to PATH environment variable.
18 [create] flatc is installed to /root/go/bin, please ensure that it is added to PATH environment variable.
19 [create] protoc-gen-secv is installed to /root/go/bin, please ensure that it is added to PATH environment variable.
20 [create] Create tRPC project `ai_play_model` post process: succeed! (" '▽' ")
21 sgameproto/ai_play_model2/lobby_require_invite
22 for x in $(find ./${dir} -name *.proto); do protoc --go_out=paths=source_relative:. ${x}; done
23 for x in $(find ./${dir} -name *.proto); do protoc --python_out=. --mypy_out=. ${x}; done
24 Writing mypy to sgameproto/ai_play_model2/lobby_require_invite/lobby_require_invite_pb2.pyi
25 Created: sgameproto/ai_play_model2/lobby_require_invite/__init__.py
26 Found .proto files in sgameproto/ai_play_model2/lobby_require_invite, running trpc create
27 sgameproto/ai_play_model2/lobby_require_invite/lobby_require_invite.proto
28 [create] Create tRPC project `lobby_require_invite` post process: succeed! (" '▽' ")
29 [master 8b1080b] auto-build
30 12 files changed, 3537 insertions(+)
31 create mode 100644 sgameproto/ai_play_model2/__init__.py
32 create mode 100644 sgameproto/ai_play_model2/ai_play_model.pb.go
33 create mode 100644 sgameproto/ai_play_model2/ai_play_model.trpc.go
34 create mode 100644 sgameproto/ai_play_model2/ai_play_model_mock.go
35 create mode 100644 sgameproto/ai_play_model2/ai_play_model_pb2.py
36 create mode 100644 sgameproto/ai_play_model2/ai_play_model_pb2.pyi
37 create mode 100644 sgameproto/ai_play_model2/lobby_require_invite/__init__.py
38 create mode 100644 sgameproto/ai_play_model2/lobby_require_invite/lobby_require_invite.pb.go
39 create mode 100644 sgameproto/ai_play_model2/lobby_require_invite/lobby_require_invite.trpc.go
40 create mode 100644 sgameproto/ai_play_model2/lobby_require_invite/lobby_require_invite_mock.go
41 create mode 100644 sgameproto/ai_play_model2/lobby_require_invite/lobby_require_invite_pb2.py
42 create mode 100644 sgameproto/ai_play_model2/lobby_require_invite/lobby_require_invite_pb2.pyi
```

仓库端：

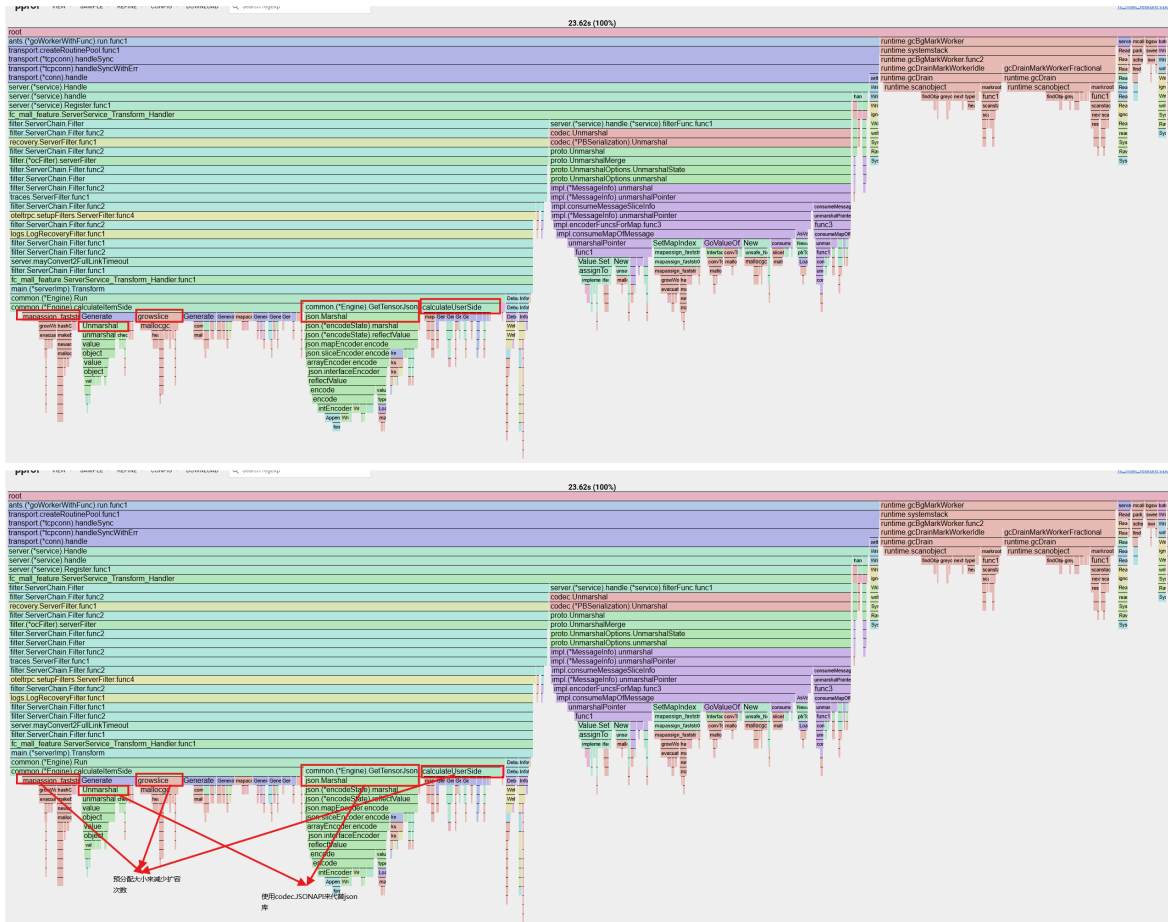
根目录ai_play_model：

文件名	最后更新于	最后的提交
-	-	-
lobby_require_invite	13 分钟 以前	auto-build
__init__.py	13 分钟 以前	auto-build
ai_play_model.pb.go	13 分钟 以前	auto-build
ai_play_model.proto	15 分钟 以前	test
ai_play_model.trpc.go	13 分钟 以前	auto-build
ai_play_model_mock.go	13 分钟 以前	auto-build
ai_play_model_pb2.py	13 分钟 以前	auto-build
ai_play_model_pb2.pyi	13 分钟 以前	auto-build
pb.py	15 分钟 以前	test
rpc.py	15 分钟 以前	test
rpc_stream.py	15 分钟 以前	test

子目录lobby_require_invite：

1. __init__.py	14 分钟 11 秒	auto-build
2. lobby_require_invite.pb.go	14 分钟 11 秒	auto-build
3. lobby_require_invite.proto	16 分钟 11 秒	test
4. lobby_require_invite.pb.go	14 分钟 11 秒	auto-build
5. lobby_require_invite_mock.go	14 分钟 11 秒	auto-build
6. lobby_require_invite.pb2.py	14 分钟 11 秒	auto-build
7. lobby_require_invite.pb2.pyi	14 分钟 11 秒	auto-build

2. fc_mall_feature优化设计：



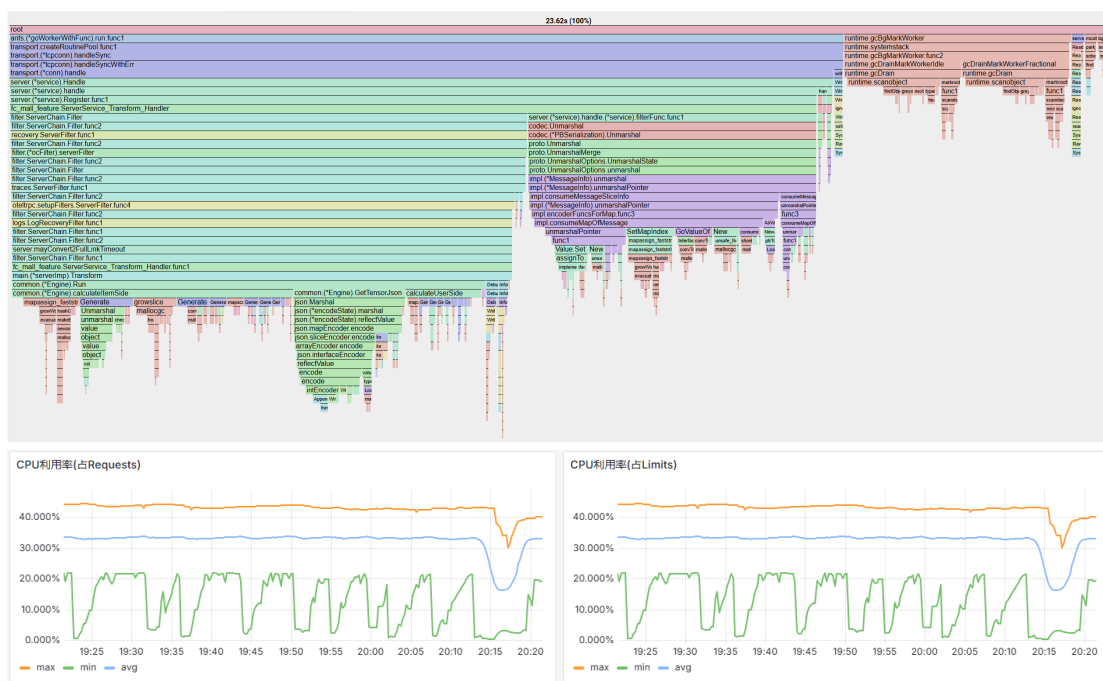
- a. calculateItemSide()和calculateUserSide()函数中的mapassign_faststr、growslice，表示map的赋值和切片扩容的cpu占比，究其原因是因为代码中的append()语句，因此我选择的优化方法是使用预分配大小，来减少扩容次数，下面给出修改后的calculateItemSide()代码：

```

1 func (e *Engine) CalculateItemSide(
2     ctx context.Context, itemFeatures []*fc_mall_feature.ItemFeature,
3     userFeature *fc_mall_feature.UserFeature, userSideInner
4     map[string]interface{},
5 ) map[string][]interface{} {
6     result := make(map[string][]interface{})
7     userData := userFeature.Data
8     for _, itemFeature := range itemFeatures {
9         itemData := itemFeature.Data
10        inner := make(map[string]any)
11        for _, def := range e.ItemSide {
12            // 预分配
13            inputs := make([]any, 0, len(def.Inputs)+len(def.InnerInputs))
14            for _, input := range def.Inputs {
15                if v, ok := itemData[input]; ok {
16                    inputs = append(inputs, v)
17                } else {
18                    inputs = append(inputs, userSideInner[input])
19                }
20            }
21            for _, input := range def.InnerInputs {
22                if v, ok := inner[input]; ok {
23                    inputs = append(inputs, v)
24                } else {
25                    inputs = append(inputs, userSideInner[input])
26                }
27            }
28            kvList, err := def.Generator.Generate(def.Name, inputs)
29            if err != nil {
30                log.Errorf(ctx, "calculate item side %v error: %v",
31                    def.Name, err)
32            }
33            for _, kv := range kvList {

```

- b. 对于GetTensorJson()和HotSaleGenerator.Generate()中的json.Unmarshal和json.Marshal，可以采用codec.JSONAPI来进行优化，即改为codec.JSONAPI.Unmarshal和codec.JSONAPI.Marshal，前者性能优于后者的原因为：
- codec.JSONAPI使用对象池（iteratorPool.Put/Get）复用迭代器，大幅减少内存分配。通过cfg.BorrowIterator和cfg.ReturnIterator实现迭代器的“池子取放”，避免高频创建临时对象，降低了垃圾回收频率；
 - codec.JSONAPI采用流式解析（iter.nextToken逐令牌读取），仅处理必要数据。nextToken跳过空白字符后直接读取有效内容，避免了全量扫描；而json标准库会先checkValid遍历整个数据验证语法，然后再调用unmarshal()进行解析，相当于就两次扫描了；
 - json标准库对输入数据会先checkValid，增加函数调用开销；
- c. 放到现网环境测试后得到此时的火焰图和cpu利用率貌似影响不大：



下一步的想法是优化go的protobuf解析的性能，因为这一块在火焰图占比较大。

今日所遇问题：

- 问题：执行sgameproto后置流水线中的脚本后都出现了“这个脚本输出已经位于 'master' 您的分支与上游分支 'origin/master' 一致”，既然都一致了，那岂不是后续git diff HEAD^ HEAD都检测不到变更了？
- 原因：脚本的输出代表的是本地 master 分支的提交历史与远程 origin/master 分支完全同步，这与目录内容变化并不冲突。比如：假设分支提交历史为：提交A—提交B（修改了目录dir1）—提交C（当前HEAD）。当脚本运行时：
 - git fetch + git checkout后，本地master指向提交C，与远程一致->此时就会输出“您的分支与上游分支'origin/master'一致”；
 - git diff HEAD^ HEAD 比较的是提交B与提交C的差异（即提交C的变更），包含目录修改，因此这里得到的目录变更与输出“您的分支与上游分支'origin/master'一致”无关；

明日待办：

1. 目前fc_mall_feature的优化方案好像对实际的cpu利用率没啥影响，后面还要继续深挖；